.curt: A Conversations-First, LLM-Native File Format with Executable Conformance for AI Chat Portability

Jordan Newell
Independent researcher

November 1, 2026

Abstract

The conversation histories users accumulate with large language model services are exported, when they can be exported at all, as undocumented, platform-internal JSON whose structure each vendor defines ad hoc and changes without notice. Legal analysis of GDPR Article 20 has established that the right to data portability fails in practice without genuinely interoperable formats [1, 2]. The 2025–2026 literature has begun to test GDPR rights against generative AI — through compliance surveys of model internals [13], conceptual work on generated data [14], and agentic-AI analyses of the EU Data Act [15] — but no scholarly work treats the conversation archive itself as a portability problem. The one open export specification published outside academia — a nonprofit's conversation-history schema [16] — defines a generic sender/text/channel transcript with no LLM-native semantics, no validation rules, and no conformance testing. We present .curt, a file format and reference implementation filling this gap: a frozen-core JSON envelope — strict whole-file validation, single-file-per-conversation, full-fidelity content including inline binary payloads — extended by a named, additive extension registry, and graded by tiered, machine-executable conformance vectors (L1 CORE through L4 per-extension) run in continuous integration. The specification, reference implementation, JSON Schema, and test vectors are MIT-licensed and published as `curt-format` on npm. We situate .curt against vendor exports, training corpora formats, agent transport protocols, formal containers (vCon, mbox), and memory-interoperability efforts (PAM, W3C), and we discuss honestly the governance model — a single maintainer with self-reported conformance — and its precedents.

Artifact: `https://dotcurt.org/` · `npm install curt-format` · MIT

Keywords: data portability; GDPR Article 20; large language models; file format design; conformance testing; personal data archives

1 Introduction

Every major LLM chat platform holds its users' conversation history in a private structure and exposes it, if at all, through a one-off export facility designed for archiving rather than re-use. The exports share three properties: they are undocumented (no schema, no normative description), platform-internal (field names and nesting mirror implementation details, not a data model), and unstable (they change without notice because nothing external depends on them contractually).

The consequences are concrete. OpenAI's ChatGPT export centers on a `conversations.json` in which a conversation is a tree: messages form a directed structure navigated by a "current node" pointer, with each node carrying role, content parts, and recursion metadata. Anthropic's Claude export flattens conversations into message lists joined by parent UUID chaining. Google's Gemini arrives inside a Takeout bundle as activity logs, not conversations at all. Each import path is different enough that every

"bridge" tool — the scripts and apps that move conversations between platforms — reimplements parsing from observations of a moving target. None of the exports carry a version. None can be validated: a consumer faced with a malformed file has no rule for detecting it, and typically half-imports whatever parses.

This is the **export trap**: the data is legally yours and practically unreachable. Portability engineering that starts from vendor exports inherits their instability, because there is nothing stable to target. The market's response is visible and measurable: a micro-industry of bridge tools — browser extensions converting chats across sixteen or more AI sites [18], offline command-line converters, round-up guides — each reimplementing its own parser against an undocumented, unversioned target, and institutional archivists documenting the exports' quirks by hand [17]. Every one of them converts *away* from the platforms (to PDF, Word, Markdown); none converts *between* them, because no target schema exists that a second platform could adopt.

This paper presents .curt (dot-curt), an open, single-file format for AI chat conversations designed as the stable target. Its design position is **conversations-first, LLM-native, and conformance-tested**:

- **Conversations-first.** One file is one conversation. There is no manifest, no account wrapper, no activity log to interpret; a directory of conversations is a directory of files, and bulk transfer is a zip of them.
- **LLM-native.** The core data model carries what LLM conversations actually contain — roles including tool results, model identifiers, timestamps, and full-fidelity content in which base64 images and tool payloads ride verbatim — with economics and multi-agent provenance available as named extensions rather than bolted-on vendor fields.
- **Conformance-tested.** A tiered vector suite (27 executable cases) defines what "conforming" means per level, and the reference implementation executes the suite in CI. Claims are graded against vectors, not prose.

The format is deliberately small: a frozen core of four envelope fields and five conversation/message fields, strict whole-file validation, and an additive extension registry as the only forward-motion mechanism within version 1. It is MIT-licensed and published as the zero-dependency package `curt-format` (v1.2.0, the first npm release) with a normative specification, a non-normative JSON Schema, and round-trip fixtures.

Contributions. (1) A problem statement for LLM chat portability grounded in the portability-rights literature, cross-analyzed against the 2025–2026 scholarship: the legal literature has reached generative AI, but the specific slot — conversation archives as a data class requiring an interoperable file format — remains unclaimed. (2) A survey of the adjacent landscape — vendor exports, training formats, agent transports, formal containers, memory-first efforts, and the one open export specification — and of the specific gap each leaves open. (3) The .curt design: frozen core, strict rejection semantics, unknown-field tolerance, extension registry, tiered executable conformance, deterministic bulk naming. (4) An honest account of single-maintainer governance with self-reported conformance, including the precedents that succeeded the same way and the one that did not.

2 The right that needs formats

GDPR Article 20 grants a data subject the right to receive personal data "in a structured, commonly used, machine-readable format" and to have it transmitted to another controller. Recital 68 makes the intent

sharper, speaking of portability between systems on the basis of **interoperable formats**. The recital is the load-bearing one for our purposes: the right is not satisfied by any machine-readable dump, but by data that another system can actually take in.

The legal literature has converged on the conclusion that this condition fails empirically. De Hert et al. trace the right's legislative history and its structural dependence on format choices that controllers make unilaterally [1]. Syrmoudis, in the sharpest formulation, shows that the portability right underperforms its competition-and-control goals precisely because "portable" data without actual interoperable formats produces *ex situ* access to inert files rather than usable migration [2]. The OECD's review of data portability initiatives across services and platforms reaches the same practical conclusion and identifies common formats and interoperability frameworks as the policy lever [3].

Article 20 scholarship has historically discussed health records (via related regimes), smart-meter data, and social-graph portability under the Digital Markets Act. A 2025–2026 wave has now reached generative AI, and we engage it directly rather than around it. The heaviest-cited work in the slot, Feretzakis et al.'s compliance survey [13], analyzes Articles 15, 16, 17, and 22 as applied to data *inside model weights* — unlearning, explainability, differential privacy — and defines Article 20 in its rights overview without analyzing it; conversation archives, export formats, and interoperability are absent from its 26 pages. Matsumi [14] asks whose data an AI output *is*, conceptually, without reaching the archive question. Beduschi [15] extends the portability discussion through the EU Data Act for agentic AI acting on the user's behalf — agent-side, not archive-side. The pattern across all three strands: the data class users actually hold — the conversation archive, a class that did not exist when the GDPR was drafted and is now among the most intimate records a person keeps — is never treated as a portability problem requiring an interoperable file format. The slot **Article 20 × LLM chat archives** is unclaimed, and now demonstrably so: the nearest neighbors are named, and the gap between them is exactly where this paper sits. It does not attempt to fill the slot legally; it fills the precondition the legal analysis identifies: an interoperable format for the data class, published openly, with an executable definition of conformance so that "supports .curt" is a testable claim rather than a marketing one.

We note one more motivation beyond the GDPR's territorial scope. Platform churn is a practical certainty in this market: services shut down, accounts are banned, pricing tiers change, and users migrate. A conversation archive that only a single vendor's current exporter can produce, and only its current importer can read, is fragile in exactly the way Syrmoudis describes, regardless of jurisdiction.

The legal analysis also fixes the requirements on the engineering side, and it is worth making them explicit because they are stricter than they first appear. For Article 20 to bind practice rather than aspiration, an "interoperable format" for LLM conversations must supply at least four things: a **published specification** that a third party can implement without observing a vendor's behavior; a **schema or validation rule set** that makes well-formedness decidable, so a receiving controller knows whether it has received what it was told it would; **stability guarantees**, so a file written today parses after the exporting platform or the importing tool changes; and **an executable definition of conformance**, so that a controller's claim to accept the format can be tested rather than trusted. No vendor export satisfies any of the four. The design in §4 is .curt's answer, item by item, to this list.

3 Landscape

We survey six families of adjacent work and the gap each leaves.

3.1 Vendor exports

ChatGPT (`conversations.json`, tree-structured), Claude (message lists with parent-UUID chaining), and Gemini (Takeout activity logs) are discussed in §1. Common properties: undocumented, unversioned, platform-internal, no schema to validate against, no import path on the vendor side. They are the problem statement, not part of the solution space.

3.2 Training-corpus formats

The fine-tuning ecosystem standardized early on JSONL line formats — ShareGPT's `conversations` array and OpenAI-style and Cohere-style chat JSONL. These are purpose-built for training: role/content pairs, aggressively normalized. They are lossy in exactly the dimensions archives care about: attachments are dropped or referenced externally, tool calls are flattened or elided, model metadata and timestamps are optional at best, and multi-modal content is reduced to text. They also have no validation story beyond "the trainer eventually errors." Training formats answer "what can a model learn from"; the archive question is "what actually happened," and the answer needs the fields training throws away.

3.3 Agent transport protocols

The Agent-to-Agent protocol (A2A) and the Model Context Protocol (MCP) standardize *live* inter-agent and agent-tool communication [10, 11]. Persistence is explicitly out of scope for both: a transcript exists for the duration of a session and its canonical serialization is the transport's own event stream, not a durable document. They are complementary rather than competing — an agent framework that speaks MCP still needs a file format for the conversations it produces. A survey of the MCP ecosystem [11] catalogs dozens of client and server implementations — and not one durable conversation-archive format among them.

3.4 Formal containers

vCon (draft-ietf-vcon-vcon-core-05) is the closest standards-track relative: a JSON container for conversational data, in IETF standardization, with a working group and a dialog data model [4]. Two things distinguish it from what LLM chat archives need. First, its home community is human/real-time communications — voice and video calls, messaging — and its data model has no LLM-native semantics: no assistant/user/system/tool role vocabulary tied to model inference, no tool-call linkage, no model metadata. Its charter explicitly scopes chat logs in, so the fair statement is not that vCon cannot represent chat, but that representing it requires application-private conventions atop a container designed for another domain. Second, conformance: vCon ships no executable test vectors, so implementation claims are checked by reading. We also read vCon's own evolution as evidence for .curt's design bet: the draft deprecates the top-level `vcon` version field in favor of an extension mechanism — the frozen-core-plus-registry pattern — confirming that containers of this kind are pushed toward additive extension rather than version churn.

mbox (RFC 4155) is the archival precedent worth keeping in view [8]: a decades-old, boring, single-file-per-folder container that outlived every proprietary mail format around it. Its lesson is the container-not-content pattern: the format standardizes envelopes and framing and stays out of content semantics. HAR (HTTP Archive) extends the same pattern to web traffic. .curt adopts the pattern deliberately.

3.5 Memory-first efforts

PAM (Portable AI Memory, v1.0) is the most serious peer effort [5]. Credit where due, and specifically: PAM has a named maintainer (Daniel Gines), published governance and semver policy pages, and real technical substance — Ed25519 signatures, W3C Decentralized Identifiers, RFC 8785 JSON canonicalization, and shipped importers for OpenAI, Anthropic, and xAI exports. Its gap relative to conversation archiving is orientation: PAM is memory-first. Conversations appear as an index plus companion files in service of a memory graph, not as first-class, self-sufficient single-file archives; and PAM defines no conformance levels, so implementation claims are not testable against vectors. The two projects are, at present, answers to different questions. The memory lane is, moreover, academically active: Ravindran's Portable Agent Memory protocol [19] presents cryptographically verified memory transfer across heterogeneous AI agents, with injection-resistant framing; and Brcic's analysis of AI memory as an infrastructure-layer lock-in problem argues that portability regulation must reach memory graphs [20] — the conversation archive this paper addresses sits directly downstream of that lock-in. That sharpened contrast is itself evidence for the gap this paper fills — the adjacent problem has scholarly work and open protocols; the conversation archive has neither.

The **W3C AI Agent Memory Interoperability Community Group**, chartered in June 2026, confirms institutional interest in the space but has produced no file format [6]. The field is open.

3.6 Open export specifications

The nearest peer effort is not academic and was easy to miss: the **Data Transfer Initiative (DTI)** — a nonprofit devoted to data portability engineering — publishes an "AI Conversation History" schema in a public schema registry (November 2025, revised May 2026) [16]. Its motivation is ours stated by a practitioner body: "Your conversations with your AI tools and services are your personal data," and its goal is switching AI services "without losing the history, context, and preferences you've built." The specification itself is a JSON transcript model: conversation-level `details` (creator, start and end times, modality), an `identifiers` array of named participant identities, and `messages` carrying `sentAt`, `sender`, `text`, and `channel`, with the rule that every sender must appear among the identifiers.

We credit it as the first open specification aimed at exactly the lock-in problem of §1, and we distinguish it precisely, because the distinction is the contribution of this paper. The DTI schema is a *registry entry*: no version number, no stated license on the registry page, no validation semantics, no conformance vectors, no tooling, and a single listed implementation. Its data model is a lowest-common-denominator transcript — a generic `sender/text/channel` triple with an identity-resolution rule — with no role vocabulary tied to model inference, no tool calls, no model metadata, no attachment story, and no bulk convention. Nothing in it is *wrong*; it simply answers the transcript-portability question ("who said what to whom, when") and stops before the LLM-native question ("what did the models do, with which tools, at what cost"). That a portability-focused standards body independently converged on the problem is evidence the problem is real; that its answer stops where .curt's design begins is evidence the design space was open.

4 The .curt design

4.1 Envelope

A `.curt` file is UTF-8 JSON with a four-field envelope:

```
{
```

```
    "curt": "curtis-conversation",
    "version": 1,
    "generator": "your-app@1.2.3",
    "conversation": { }
}
```

`curt` is the magic field, matched exactly. `version` is the format version, matched exactly — the string `"1"` is a rejection, a rule that costs nothing and eliminates an entire class of lenient-parser bugs. `generator` is provenance ("writer-name@version") that readers **must ignore**: presence, absence, or value never affects parsing, which keeps the format from growing de facto required vendor stamps. `conversation` carries `id`, `title`, `createdAt`, `updatedAt` (epoch milliseconds), and `messages`; each message carries `id`, `role` (`user`, `assistant`, `system`, `tool`), `content`, `timestamp`, and an optional `model` (provider model id, typically on assistant messages). That is the entire core: four envelope fields, five conversation fields, five message fields.

Two envelope decisions deserve defense. First, **epoch milliseconds** for time: no calendar semantics, no timezone ambiguity, trivially comparable, and tolerant at the validator level (version-1 validators accept any JSON number and must not reject fractional or negative values) so that a file written from a language with richer time types is never rejected over its representation. Second, **content as a single string**: binary payloads ride inline as base64 or data URIs, verbatim. Readers never decode, transform, or re-encode content. This is what "full fidelity" means operationally: the bytes that went in are the bytes that come out, and a small conversation with images may legitimately be a large file. The specification accordingly directs implementers to treat files as untrusted input and bound parsing by size before validation.

4.2 Strict whole-file validation

Version-1 validation is a conjunction: the body parses as JSON (tolerating at most one leading UTF-8 BOM, which writers must not emit); the magic matches exactly; `version` is exactly the number 1; the conversation and every message carry required fields with exact types. Anything else is a rejection of the whole file — never a best-effort parse that half-imports. The rationale comes directly from the failure mode of vendor exports: when a reader imports "whatever parses," corruption propagates silently and no one can define what was lost. Whole-file rejection converts silent corruption into a loud, actionable error at a known boundary. The reference implementation's `parseCurt` returns the parsed conversation or `null` — null means invalid — and its component predicates (`isCurtConversation`, `isCurtMessage`) are exported for reuse, so third-party validators share the canonical behavior rather than re-deriving it.

Strictness has a cost: one malformed message dooms its entire conversation file, and a strict reader must skip that whole file in a bulk import. The bulk convention (§4.5) restores the pragmatic middle ground at the level where it belongs — the archive — by requiring readers to import valid entries and *report* corrupted ones rather than drop them.

4.3 Unknown fields: tolerated, preserved where possible

Real conversations carry more than any core can anticipate. Writers may attach extra fields at the top level, on the conversation, and on messages. Readers must not fail on them. On round-trip, readers preserve unknown fields **where possible**.

The qualifier matters and we state it precisely. Preservation is a quality aim, not a normative guarantee: the reference implementation preserves unknown fields on the conversation object and on messages

verbatim, but rebuilds the envelope on serialization and so drops unknown top-level envelope extras. A reader that cannot represent an unknown field in its internal model will drop it too. The spec's obligation is non-failure and best-effort preservation; stronger guarantees would be unenforceable across heterogeneous implementations and would make the format harder to adopt, not safer. Unknown fields are data, never code: the specification directs implementers not to interpret, dereference, or execute anything found in them — a caution underlined by serialization-vulnerability advisories in adjacent ecosystems [12].

4.4 Frozen core + extension registry

Capability growth inside version 1 proceeds through a named extension registry, not spec edits. A writer declares the extensions it uses in an optional top-level `extensions` array of registry names. Three properties make the mechanism safe:

1. **Declaration costs nothing.** The array is itself an unknown field to old readers: never a validation input, never a reason to reject, whatever its shape.
2. **Extensions are behaviorally inert at the core.** Extension behavior lives entirely in the fields an extension reads and writes inside the conversation and its messages; an extension must not change how the core validates. Malformed extension data cannot corrupt a conforming parse.
3. **The registry is additive.** New entries never change how existing v1 files parse, and entries cannot conflict with core field names. Status moves `draft` → `stable` when at least two independent implementations round-trip the entry.

The seed registry is drawn from what implementations already round-trip informally — grandfathered as stable at seeding, since both entries predate the promotion rule, which governs entries that enter as draft: `images-inline` (stable; payloads inside `content`), `usage-metadata` (stable; per-message token counts, cost, provider), `tool-linkage` (draft; assistant `toolCalls` linked to `tool`-role results by `toolCallId`), and `agent-context` (draft; multi-agent provenance such as swarm role, leader id, skill ids). Since the first audit, a fifth entry has arrived — `consensus-receipt` (draft; signed k-of-n consensus decisions bound to the transcript evidence), submitted by the author's own project, tractum — so it exercises the mechanism rather than evidencing adoption: a new capability class entered the format, `version` stayed `1`, and no existing file parsed differently. The core/tool-role split is deliberate: the *existence* of tool results is core — an LLM conversation without them is not full fidelity — while the linkage schema is still stabilizing and so rides as a draft extension. Cost, token, and provider metadata are similarly placed: widely written, not yet universal, and therefore registry rather than core.

This is the same evolutionary pressure vCon responded to by deprecating its version field in favor of extensions (§3.4). .curt starts there: the registry is the only forward-motion mechanism inside version 1, and breaking changes — anything that would make a conforming v1 reader reject or misparse a file it previously accepted — require version 2, never a silent redefinition of 1.

4.5 Tiered executable conformance

"Conforming implementation" is made testable by a tiered vector suite, executable as plain JSON fixtures:

The suite comprises 27 cases: 13 L1 accept/reject bodies, 7 L2 naming cases, 6 extension cases (all must-accept, encoding the rule that extension presence or malformation never causes rejection), and the L3 rules with a canonical example (zip-level behavior asserted by rule rather than binary fixture, so the vectors stay plain JSON). The reference implementation executes the full suite in CI. A third party

Level	Name	Requirement	Obligation
L1	CORE	Strict v1 parse/serialize: magic + exact version; whole-file rejection; single leading BOM tolerated; unknown fields preserved where possible on round-trip; any JSON-number timestamps accepted	MUST for "conforming"
L2	NAMED	`curtFilename` implements the deterministic naming algorithm exactly	SHOULD
L3	BULK	Zip convention; readers never depend on names; corrupted entries reported, valid ones imported	MAY
L4+	per extension	That extension's vectors	MAY

claims a level by passing its vectors and recording the claim — with a link to a reproducing run — in the ecosystem's machine-readable `capabilities.json`.

The conformance design answers the question "what does it mean to support .curt?" with a per-level answer instead of an adjective. It also creates the ecosystem's honest scoreboard: at the first audit (2026-10-14), only the reference implementation passed L1 (§5).

4.6 Deterministic naming and bulk transfer

Bulk transfer is a zip of `.curt` files with no manifest requirement. Entry names SHOULD come from `curtFilename`, and readers MUST NOT depend on names for parsing or ordering — names are a convenience, never an input. `curtFilename` is specified so the name depends only on immutable fields: `createdAt` rendered as a UTC date, a slug derived from the title, and a suffix derived from the id — so the name survives title edits and later updates, orders chronologically, and collides safely. Example: `2025-10-09 fixture-conversation 001.curt`.

4.7 The Schema artifact

Alongside the normative text, the project ships a non-normative JSON Schema (draft-07) rendering of the validation rules, for editors, linters, and third-party tooling. Its non-normativity is deliberate and stated: the rules of SPEC.md are the norm, and the reference implementation is the arbiter when prose and schema disagree. A schema alone cannot express whole-file rejection semantics or round-trip obligations, and treating a schema as the specification invites the drift .curt's audit found in the wild — five vendored copies, none fully conforming (§5). The schema earns its place as an adoption accelerator for tool builders, not as a second norm.

5 Governance

We describe the governance as it is, not as a committee would describe a committee.

One maintainer. dot_curt has a single maintainer acting as BDFL. Spec, registry, and vector changes land as ordinary commits; there is no working group, no chair, no consensus procedure. This is a limitation (§6) and, at the current scale, a feature: the format moves at the speed of its problem, and the audit trail is the git history.

Self-reported conformance. Conformance claims are made by implementers, against public vectors, recorded in a public tracker, with the expectation of a reproducible run. Nothing independently audits

the reference implementation either; what makes the system honest is that the vectors are executable and public, so any claim can be checked by anyone in minutes. The first ecosystem audit (2026-10-14) graded the five consumers — all applications within the author's own Curtis ecosystem — and found none passing L1: the reference implementation is the only conforming reader, and the drift concentrates on BOM tolerance and the naming algorithm, precisely the failures the vector suite exists to catch. The tracker exists to make drift visible, and it did.

Precedents. Small-foundation formats have reached ubiquity before, in exactly this shape. JSON was controlled by Douglas Crockford until it was standardized as RFC 8259 [9]; RSS 2.0 was unforked and shepherded by Dave Winer with a frozen spec. The instructive failure is Markdown, whose ambiguity plus single-implementer drift produced the CommonMark fork — a community rebuilding the standard because the original would not move [7]. The lesson .curt takes is to freeze the core early (done: version 1's rules are frozen at release), keep extension as the only motion (done), and donate to a foundation early enough that the fork never becomes the rational move.

Licensing. The licensing story is version-scoped and stated plainly: through release 1.1.0 the spec, reference implementation, schema, and fixtures were BUSL-1.1; as of 1.2.0 (2026-10-14) they are MIT, permanently. A standard that requires permission to implement is not open, whatever its file format is; the MIT relicense was prerequisite to this paper's claims. The Curtis applications that gave rise to the format remain BUSL-1.1 — the moat is in the apps, not the standard.

6 Limitations

No cryptographic layer. PAM signs memories (Ed25519) and identifies actors via DIDs; .curt has no signing, no canonicalization, no provenance chain. For conversations — where the integrity threat is corruption, not forgery, and the archive is typically self-produced — we judge the complexity unearned today. The extension mechanism is the migration path: a `signatures` extension alongside RFC 8785 canonicalization slots in without touching the core, if and when the need materializes.

Preservation where possible. As specified in §4.3, unknown-field preservation is best-effort by design. A pipeline of readers and writers can, in the worst case, shed enrichment fields. Applications that must not lose such fields should treat the .curt file as the system of record and re-read it, rather than relying on intermediate representations.

Container, not content. .curt standardizes the envelope. It does not define content semantics — what a "summary" means, how tool arguments are typed, what a memory is. This is the mbox/HAR/vCon pattern adopted deliberately (§3.4), but it means semantic interoperability between two .curt-reading applications is real only for core fields and registered extensions; everything else is passthrough.

Governance concentration. One maintainer is a bus factor of one, and conformance is self-reported. The JSON and RSS precedents suggest this can work (§5), and the CommonMark precedent suggests what happens when it stops working. The mitigation chosen — frozen core, executable vectors, public tracker — is to make the standard's *artifacts* independent of any maintainer's continued attention.

Parallel standardization. DTI's conversation-history schema [16] now addresses the same problem statement from the portability-engineering side (§3.6). Two open specifications for AI conversation export could fragment exactly the bridge-tool ecosystem the format is meant to unify. The differences are concrete — LLM-native semantics, validation, conformance — and convergence is a live option in either direction: a .curt interop profile of the DTI identity model, or LLM-native extensions registered against DTI's schema. Competition between open specifications is healthier than competition between proprietary exports, but it is still fragmentation, and it is the landscape's most likely near-term change.

Adoption is unproven. The honest scoreboard: as of the first audit, zero consumers depend on the

`curt-format` package and none of the five consumers is independent of the author; the in-ecosystem implementations — the author's Curtis applications — vendor or hand-roll their parsing, and only the reference implementation passed L1. One consumer has since joined through the registry process — tractum, the author's own project, writer of the `consensus-receipt` extension — which demonstrates the registry mechanism but is not independent growth: still no independent adoption and no third-party conformance run. The format's technical claims are executable and tested; its ecosystem claims are aspirations with a tracker. The intended sequence — MIT license, then package adoption by consumers, then third-party implementations — has begun to produce evidence: on 2026-11-01 the first package-dependency consumers, the author's own conversion tools (curt-vcon, curt-convert, curt-wildchat), published to npm declaring `curt-format` as their dependency. This is not independent adoption — every consumer remains the author's — and no third-party conformance run exists; but the tracker records both facts, which is the mechanism working as designed.

7 Conclusion and artifact

We presented .curt, a single-file, conversations-first, LLM-native conversation format with a frozen core, an additive extension registry, and tiered executable conformance, together with an account of the gap it fills: vendor exports are undocumented and unstable, training formats are lossy, agent transports do not persist, standards-track containers lack LLM-native semantics and executable vectors, memory-first efforts index conversations rather than archive them, and the one open export specification — a portability nonprofit's schema registry entry — stops at a generic transcript with no validation and no conformance story. The portability-rights literature establishes that rights without interoperable formats fail, and the 2025–2026 scholarship reaches generative AI without ever reaching the conversation archive; .curt is the format-side contribution for that unclaimed data class. The invitation to convergence with parallel efforts (§6) stands open.

Artifact. The specification (SPEC.md), extension registry (EXTENSIONS.md), conformance vectors (`conformance/`), ecosystem tracker (`capabilities.json`), JSON Schema, and zero-dependency reference implementation are MIT-licensed at `https://github.com/JordanNewell/dot_curt` and published as `curt-format@1.2.0` on npm; the spec site is `https://dotcurt.org/`. Vectors are plain JSON and runnable without the reference implementation; we invite independent implementations and conformance reports.

Preparation disclosure

Literature discovery, citation verification, and first-draft prose for this paper were produced with AI assistance under the author's direction. Every source cited was retrieved and verified against the original; the verification method is documented in a public evidence report accompanying the artifact; and all claims, decisions, and accountability rest with the author.

References

[1] P. De Hert et al., "The right to data portability in the GDPR: towards user-centric interoperability of online services," *Computer Law & Security Review* 34(2), 2018. `https://www.sciencedirect.com/science/article/pii/S0267364917303333`

[2] E. Syrmoudis et al., "An Empirical Analysis on the Effectiveness of GDPR Art. 20 'Right to Data Portability'," *PoPETs* 2021(3), 351–372. https://petsymposium.org/popets/2021/popets-2021-0051.pdf

[3] OECD, *Mapping data portability initiatives, opportunities and challenges*, OECD Digital Economy Papers No. 321, 2021. https://doi.org/10.1787/a6edfab2-en

[4] D. Petrie, "vCon Core," Internet-Draft draft-ietf-vcon-vcon-core-05, October 2026, and WG charter. https://datatracker.ietf.org

[5] Portable AI Memory (PAM) v1.0. https://portable-ai-memory.org · https://github.com/portable-ai-memory

[6] W3C AI Agent Memory Interoperability Community Group, charter, June 2026. https://www.w3.org/community/ai-agent-memory-interop

[7] J. Atwood, "Standard Markdown is now Common Markdown," 2014. https://blog.codinghorror.com/standard-markdown-is-now-common-markdown

[8] E. Hall, "The application/mbox Media Type," BCP 106, RFC 4155, 2005. https://www.rfc-editor.org

[9] T. Bray (ed.), "The JavaScript Object Notation (JSON) Data Interchange Format," RFC 8259, 2017. https://www.rfc-editor.org

[10] Google, "Agent2Agent (A2A) Protocol specification." https://a2a-protocol.org

[11] A. Ehtesham et al., "A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)," arXiv:2505.02279, 2025. https://arxiv.org/abs/2505.02279

[12] GitHub Security Advisory GHSA-c67j-w6g6-q2cm, LangChain serialization vulnerability. https://github.com/advisories/GHSA-c67j-w6g6-q2cm

[13] G. Feretzakis, E. Vagena, K. Kalodanis, P. Plevraki, D. Kalles, A. Anastasiou, "GDPR and Large Language Models: Technical and Legal Obstacles," *Future Internet* 17(4):151, 2025. https://www.mdpi.com/1999-5903/17/4/151 · DOI 10.3390/fi17040151

[14] H. Matsumi, "Generating Personal Data and the GDPR: Conceptualizing, Analyzing, and Recognizing Generated Personal Data," *Canadian Journal of Law and Technology* 23(2), 2026. https://digitalcommons.schulichlaw.dal.ca/cjlt/vol23/iss2/5

[15] A. Beduschi, "Data protection in the era of agentic artificial intelligence," *Computer Law & Security Review* 61:106342, 2026. DOI 10.1016/j.clsr.2026.106342

[16] Data Transfer Initiative, "AI Conversation History," schema registry entry, November 2025, revised May 2026. https://schemas.pub/schemas/24 · https://dtinit.org/open · source: https://github.com/dtinit/schema-refs

[17] Baruch College (CUNY) Libraries, guide to archiving AI chat transcripts (documents ChatGPT's ZIP-of-JSON and Claude's single-JSON exports). https://guides.newman.baruch.cuny.edu/c.php?g=1527743&p=11443010

[18] Bridge-tool ecosystem, market evidence: AI Exporter (Chrome extension, 16+ AI sites), https://chromewebstore.google.com/detail/kagjkiiecagemklhmhkabbalfpbianbe · ai-chat-md-export (offline CLI), https://github.com/sugurutakahashi-1234/ai-chat-md-export · "5 ways to export AI conversations," https://backrun.co/blog/5-ways-to-export-ai-conversations-to-pdf-word-or-markdown

[19] S. K. Ravindran, "Portable Agent Memory: A Protocol for Cryptographically-Verified Memory Transfer Across Heterogeneous AI Agents," arXiv:2605.11032, 2026. https://arxiv.org/abs/

2605.11032 · DOI 10.48550/arXiv.2605.11032

[20] M. Brcic, "The Memory Wars: AI Memory, Network Effects, and the Geopolitics of Cognitive Sovereignty," arXiv:2508.05867, 2025. https://arxiv.org/abs/2508.05867
Legal texts: GDPR Article 20 and Recital 68, https://gdpr-info.eu.